

Cumulative Exact Mixture Search for Password Recovery

Public technical report · Public draft 0.4

3 October 2026

Abstract

This paper explains how a recovery search can change its assumptions without losing the meaning of earlier work. Its purpose is to make ranking, revision, and completion accounting reviewable outside the code.

The system gives alternative password constructions explicit weights, adds the contributions of every construction that produces the same byte string, and orders distinct strings by their resulting model probability. It represents sets of strings as shared, counted graphs. When the model changes, it subtracts accepted historical coverage from the new model by string membership, preserving the new ranking of everything that remains. A campaign controller records assignments and acknowledgments against immutable plans, targets, and checker policies.

These are separate claims about ranking, set membership, and execution. Exact arithmetic does not make subjective weights accurate. A compact graph does not make an arbitrarily broad model cheap. A durable acknowledgment does not make a fallible checker infallible.

The public implementation has passed small exact oracles, large symbolic fixtures, local synthetic cryptographic controls, and clean-checkout software tests on Linux and macOS. Those results support the tested behavior. They do not establish universal crash safety, production GPU performance, or the probability of recovering an unknown password.

Keywords: password recovery; mixture models; finite languages; symbolic set difference; durable coverage

1 Why revisions matter

An authorized password recovery may start with several remembered structures and uncertain interpretations. Some details may be wrong or incomplete. A useful search therefore needs to accommodate changing beliefs about fragments, lengths, separators, capitalization, and transformations.

The hard operational question is not simply how many strings a generator can produce. It is how to choose the next useful checks while retaining an exact account of earlier work. A count such as “ten billion checked” is insufficient unless it identifies the bytes, target, checker, and completed region.

Probability-guided password guessing already exists. PCFG work generates password structures in probability order; OMEN uses an ordered Markov enumerator. This project does not claim that ordinary password tools only search alphabetically or cannot use broad models. See Weir et al. 2009 [1] and Dürmuth et al. 2015 [2].

The design examined here combines three requirements:

1. Rank distinct output strings by the sum of their model explanations.
2. Preserve accepted coverage when constructions and weights change.
3. Make each exclusion traceable to an immutable plan and an explicit evidence policy.

Forensic preservation remains a prerequisite. Work uses copies of identified inputs; a header identifier alone cannot establish that the correct encrypted payload accompanies it. A perfectly reproduced search against the wrong target remains the wrong search.

2 What the ranking means

2.1 A distribution over byte strings

Let x be a finite byte string and r a model revision. Each hypothesis h defines a normalized distribution $p(h, x)$, and has a normalized mixture weight $w(r, h)$. The model probability of x is:

$$p_r(x) = \sum_h w(r, h)p(h, x) \quad (1)$$

$$\sum_h w(r, h) = 1, \quad \sum_x p(h, x) = 1$$

If a hypothesis itself has several derivations of x , their contributions must also be added. The compiler ranks strings, not derivation paths.

“Exact” means exact evaluation of this specified finite rational model, within the implementation’s numerical and resource limits. It does not mean that $p_r(x)$ is the true probability of the owner’s original choice. Evidence metadata and confidence labels do not become numbers automatically; weights are explicit modeling decisions.

There is also uncertainty outside the model. Normalizing all admitted strings to total mass one does not establish that the real password is among them. If the model is interpreted as conditional on admitted support, its reported mass must retain that interpretation.

2.2 A complete small example

Consider two deliberately public, synthetic hypotheses:

- H1 emits **ab** or **ac**, each with probability 1/2.
- H2 emits **ab** with probability 1/4 and **bc** with probability 3/4.

Initially, give H1 weight 3/5 and H2 weight 2/5.

Table 1: Summed string probabilities under the initial mixture.

String	Contribution from H1	Contribution from H2	Total
ab	3/10	1/10	2/5
ac	3/10	0	3/10
bc	0	3/10	3/10

The order is **ab**, **ac**, **bc**. The two explanations for **ab** reinforce the same string; it is emitted once. Equal scores are resolved by bitwise lexicographic order, so **ac** precedes **bc**.

Now suppose **ab** has an accepted, durably recorded negative result. Later, change the mixture weights to H1 = 1/5 and H2 = 4/5.

Table 2: Revised ranking after accepted history excludes **ab**.

String	Revised probability	Eligibility
bc	3/5	Remains
ab	3/10	Excluded by accepted history
ac	1/10	Remains

The new search is **bc**, **ac**. The old completion referred to **ab** under the old plan; it cannot be reinterpreted as “position zero in whichever plan is current.”

The remaining strings retain scores 3/5 and 1/10. Their total is 7/10. Renormalizing them would give 6/7 and 1/7 without changing order, but the stored plan preserves the original model scores. Interpreting that normalization as a posterior requires trustworthy negative evidence. A checker with possible false negatives does not justify treating excluded strings as impossible.

The public example verifier^{E1} reproduces both complete orders, subtraction, and bounded preparation of the highest remaining score band. It uses simulated coverage only; no password check is performed.

2.3 The scope of the priority claim

For a fixed model and K equally costly, reliable checks, choosing the K distinct eligible strings with greatest model probability maximizes the probability mass assigned to those checks. If a selected string has lower mass than an omitted one, exchanging them improves the total. Repeating that exchange gives the descending order.

This is a statement about the supplied distribution. It does not establish calibration, optimal wall-clock scheduling with variable costs, or recovery within a chosen budget. GPU execution is also parallel: ranked batch submission does not mean physical checks finish in rank order.

3 How the engine represents and subtracts sets

3.1 Shared continuation graphs

A language here is a set of finite byte strings. A deterministic acyclic graph represents that set through byte-labeled edges and accepting endpoints. “Deterministic” means at most one outgoing edge for each byte; “acyclic” means a path cannot loop.

Nodes with identical accepting status and identical labeled continuations are shared. For example, many prefixes can lead to the same suffix choices without storing those choices repeatedly. This is hash-consing: a hash accelerates lookup, while full structural equality decides whether two nodes are equal. A hash collision is not itself a declaration of equality.

Each node stores the number of accepted suffixes below it. Let $\text{accept}(v)$ be one when node v is accepting and zero otherwise:

$$\text{count}(v) = \text{accept}(v) + \sum_{(b,u) \in \text{edges}(v)} \text{count}(u) \quad (2)$$

Outgoing first bytes distinguish the branches, so those counts can be added without double counting. Counts allow the emitter to skip whole subgraphs when seeking an ordinal position. It need not emit every earlier string.

Compactness depends on repeated structure. A huge regular set can be small; a fragmented set of unrelated strings may not be. Serialized plan size and peak preparation memory are different quantities.

3.2 Combining explanations before ranking

The input is a weighted graph. At a given output prefix, several source paths may still explain the bytes seen so far. The compiler tracks that collection of source states and their integer amplitudes, combines matching states, and adds terminal contributions.

It groups the resulting deterministic languages into **score bands**. Every string in one band has the same exact score; different bands are disjoint. Bands are ordered from highest score to lowest, and strings within a band use bitwise order. The representation can therefore describe and rank many strings without materializing a wordlist or an individual probability record for each.

The current native core uses checked unsigned 128-bit arithmetic for scores and a common denominator, with checked 64-bit counts. Inputs that exceed supported arithmetic are rejected rather than silently rounded. Python model weights accept exact integers and rational strings; floating-point weights are rejected. Exactness applies to successfully compiled supported inputs.

Sources: model semantics^{E2}, rational graph export^{E3}, and native compilation and plan validation^{E4}.

3.3 Historical coverage is a set with provenance

An immutable plan P defines an ordered sequence. An acknowledged interval $[\text{start}, \text{start} + \text{count})$ refers to that sequence under P ’s original identity. The engine converts the interval into a language using node counts and symbolic slicing.

For accepted intervals from any number of retained revisions, let I_j denote the string set of interval j :

$$C = \bigcup_j I_j, \quad E_r = \text{supp}(p_r) \setminus C \quad (3)$$

Each revised score band is intersected with the complement of C. Surviving strings keep their scores and relative order. Reimporting the same accepted completion does not enlarge the union. Temporarily removing a hypothesis does not erase its history; reintroducing it still encounters the same excluded byte strings.

Set difference is implemented by matching byte transitions and accepting states, not by approximate fingerprints or a Bloom filter. The workspace preparer also collects unreachable intermediate nodes between operations. That improves some preparation workloads but does not remove worst-case graph growth. See workspace preparation^{E5}.

The mathematical exclusion is exact for the accepted set. Whether that set should be accepted is a separate question: target identity, checker policy, receipt integrity, and historical evidence still matter. A generated preview, an unfinished assignment, or an unverified aggregate count contributes no completion by itself.

3.4 Bounded preparation and resource limits

The native `prepare` command can retain only the highest requested number of distinct score levels. It retains complete ties and records whether the remaining support is complete, together with an upper bound on the next omitted score. A partial plan must not be described as the whole search space.

This bound limits score levels, not candidate count. One equal-score band may contain trillions of strings; ordinary batch limits still determine how much is emitted or checked.

The normal model-bundle workflow currently performs full compilation. Python compiles each hypothesis and builds its complete score index before native mixture compilation. Requesting a few native score bands does not bypass those earlier costs. Continuing beyond a partial plan requires another explicit preparation and adoption step; a campaign does not automatically fill the omitted tail. See the model-bundle workflow^{E14}.

Current native guards include 600,000 language nodes, 400,000 weighted subset states, 8,000,000 cached score entries, a 64 MiB plan-file cap, and a maximum candidate length of 128 bytes. The native preparation guard also checks 90-second and 768 MiB budgets cooperatively. These do not bound the entire Python-to-native workflow, guarantee complexity, or impose exact operating-system memory ceilings.

The small-model tests compare output against exhaustive exact oracles. The inductive counting and set-operation arguments explain the intended invariants; neither those arguments nor sanitizer runs constitute a formal verification of every C++ execution.

4 Broad support and finite budgets

A model can admit far more candidates than a campaign will ever afford to check. That can be useful: uncertain details receive a low prior instead of being ruled out permanently. It works only if the probabilities and represented structure remain useful.

Branch weight is not per-string priority. Suppose two disjoint branches are uniform:

Table 3: Branch mass and per-string priority for disjoint uniform branches.

Branch	Total mass	Distinct strings	Mass per string
Broad	99/100	1,000,000,000,000	0.000000000000099
Narrow	1/100	10	0.001

Every narrow-branch string comes first despite that branch having only one percent of the mass. With overlap, contributions are summed before making the comparison. “Give the core theory 99%” does not mean “spend the first 99% of checks on that theory.”

A public symbolic test represents all twelve-letter lowercase strings: $26^{12} = 95,428,956,661,682,176$ distinct strings. That is a counted finite language, not an executed search. The test checks exact rank-budget membership without materializing those strings. This structured example establishes no general cost bound; see public tests^{E12}.

Adding a broad tail can increase source size, weighted-state combinations, score diversity, intermediate graph size, and arithmetic requirements. It can also alter the highest-ranked

strings through overlap or normalization. A successful compile of one broad model is not evidence that every broader revision will compile cheaply.

The useful review quantity is mass reached within an affordable budget. Let $T_{r,K}$ contain the first K eligible distinct strings:

$$M_r(K) = \sum_{x \in T_{r,K}} p_r(x) \quad (4)$$

This is model mass, not measured recovery probability. Rank previews should also show which hypotheses contribute, how length patterns change with K , and how sensitive the first K strings are to plausible alternative weights. Membership in overlapping branches is not a disjoint allocation of GPU time.

Weight uncertainty can initially be examined through explicit alternative profiles. A future model could integrate over uncertain weights, but doing so requires specifying a distribution and a decision objective. It is not automatically an unsolved theoretical problem.

5 Reviewing a model before changing it

The engine provides consistent consequences for a model. It cannot decide which recollection is reliable. A review should keep four things separate: the observation, its interpretation, the generating rule, and the assigned weight. Repeated experimental seed expansions are not independent recollections and should not acquire extra probability merely because they were written often.

A construction might contain a prefix, separators, two inner words, and an ending. An ending made only of symbols is still a finite string component. Changing word order is a distinct arrangement; it need not change the identity of the component choices or the relationships between them.

5.1 Components and correlations

A listed word, its capitalization, and an edit operation describe different choices. A short spelling already present in a dictionary is not automatically a deletion error. Source lengths are selected before casing and transformations; removing a character can produce unequal output lengths from equal source lengths. Penalizing the same omission again as an unexplained length deviation can count one event twice.

Selecting one separator and reusing it differs from drawing a new separator at every boundary. Sharing a case style across words differs from independently choosing each style. A joint distribution over source-length pairs can favor equality without assuming independent lengths. The construction language has shared bindings for these dependencies; accidental independence changes both support and probabilities.

Multiplying deviation penalties assumes a particular factorization. Exact fractions do not justify that assumption. Correlated deviations may instead need a shared latent choice or an explicitly weighted joint branch.

5.2 Transformations with explicit meanings

For a public illustrative source 1234, identity gives 1234, reversal gives 4321, and a US number-row shift swap gives !@\$. These are operational definitions, not observations about an unknown password. The current recipe frontend chooses one weighted transformation per emission; a list of operations is not an automatic pipeline. Generic letter-alphabet slots do not currently support all finite-slot transformations.

5.3 Sensitivity and adoption

Alternative interpretations should be named and compared under several plausible allocations. Splitting an existing branch into equivalent subbranches should preserve its total allocation unless the intended belief changes. Every extra variant should have a rationale rather than an invented confidence percentage.

Sampled candidates make consequences inspectable but do not prove calibration. The local workshop supplies structure views, exact ranked explanations, and source-pair membership queries. Overlapping explanations contribute to the same candidate score; heatmap cells can overlap and cannot simply be summed into a unique count. The author must still review

unsupported possibilities, dependence assumptions, and affordable early batches. Models remain separate private inputs.

6 From a ranked plan to recorded completion

6.1 The actual execution path

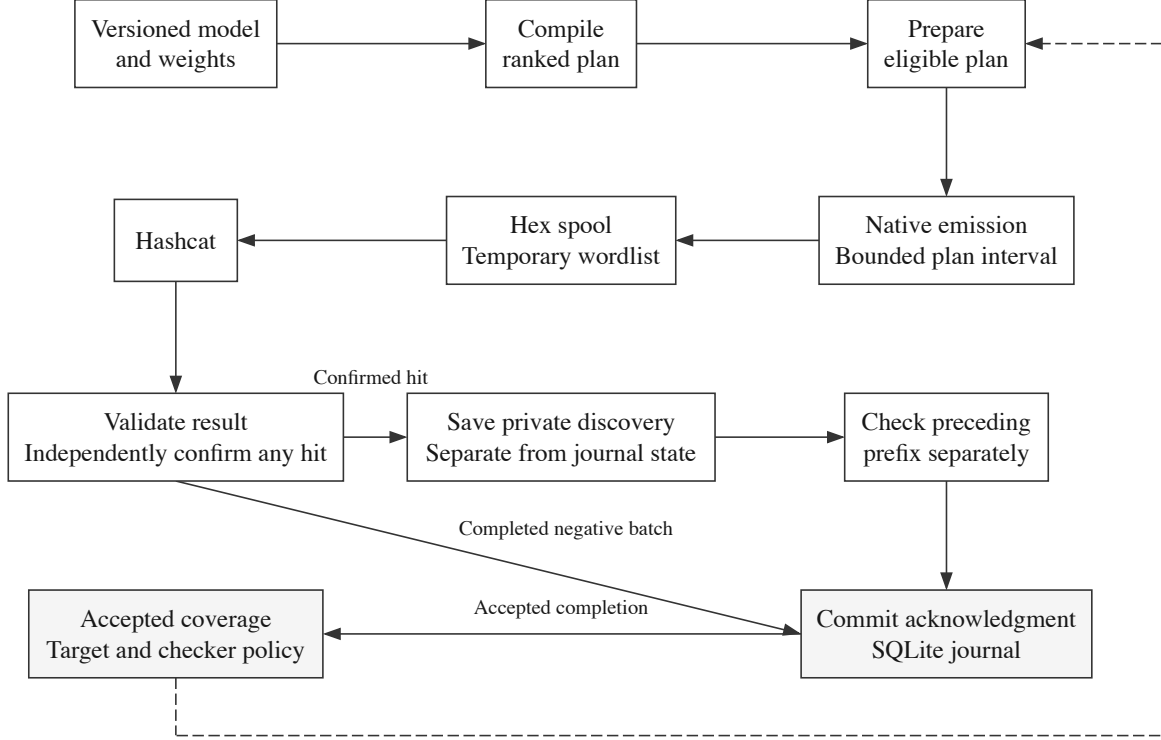


Figure 1: Ranked preparation and durable campaign accounting. A confirmed hit is saved before its preceding prefix is committed.

The sequence appears in Figure 1. Python compiles each hypothesis into a weighted automaton and a complete score index, then exports the graphs for native mixture compilation and emission. The native emitter sends length-framed bytes through a pipe to the hex spool adapter. The adapter writes a bounded temporary hex wordlist. Hashcat reads that file. It is not a continuous generator-to-Hashcat stdin stream.

The generic native language supports byte values and lengths beyond the current GPU bridge. The bridge accepts nonempty printable ASCII candidates of at most 64 bytes and rejects an unsupported batch rather than silently dropping strings. Its current maximum batch is one million candidates.

A job binds an interval to an immutable plan and target. The campaign pins the checker configuration, executable, kernel/module and adapter identities, and its qualification record. Drivers and shared runtime libraries are not fully pinned; hardware or driver changes require renewed qualification. The acknowledgment identifies the job, plan, target, submitted count, status, and consecutive negative prefix. It may contain a hit encoded as hexadecimal bytes. See GPU backend^{E6}, spool adapter^{E7}, and acknowledgment validation^{E8}.

6.2 A hit is not proof that every earlier rank finished

Suppose a batch is [a, b, c, d] and parallel GPU execution finds c. A hit at rank two does not, by itself, certify that a and b completed. Nor does it certify d.

The controller independently confirms the reported password with the native LUKS master-key-digest checker and QEMU. It saves that confirmed discovery immediately in a separate private `discoveries/` file. Before a hit receipt commits, reopening leaves the journal assignment pending and may still report no hit. Status and audit do not ingest the file as a journal hit.

The saved discovery must be considered separately when deciding to retry or reconcile the campaign; automatic migration requires reconciliation.

The controller then separately checks the preceding prefix before committing a rank-based hit receipt. Those prefix negatives still use the stated Hashcat policy. A later failure cannot justify inventing completion for an unfinished prefix, and it should not lose the already confirmed secret. See GPU worker^{E9}.

This extra prefix check can repeat candidates that the GPU already evaluated. Avoiding unsupported coverage claims takes precedence over avoiding all repeated physical checks.

6.3 Commit boundaries and interruption

The controller uses a single local owner and a SQLite journal with WAL and full synchronization settings. Event replay and checkpoints reconstruct state; audit checks the retained identities and transitions. See checkpoint controller^{E10}.

Table 4: Commit boundaries determine what can be credited on restart.

Interruption point	What remains authoritative	Restart behavior
Before any accepted completion	Persisted assignment, no new negative credit	Replay pending assignment
After checking but before receipt commit	Earlier journal state	Replay may repeat already executed checks
After durable discovery but before completion commit	Private discovery file; journal assignment still pending	Review the saved file separately; do not infer a journal hit or completed prefix
After completion commit	Committed acknowledgment and its prefix	Retain that coverage on reopen and revision

The design therefore permits **at-least-once execution** around lost results. Its coverage accounting is set-based and idempotent. It does not guarantee that a candidate is physically checked only once.

The journal’s hash chain and stored content hashes make many corruptions and inconsistencies detectable against retained identities. They are not remote execution attestation. A malicious operator could fabricate a self-consistent history; an old valid snapshot cannot establish freshness without an independent anchor. Durability also depends on the storage stack honoring synchronization. The current qualification exercised abrupt process exits, not every possible power failure, filesystem fault, or distributed failure.

6.4 What a LUKS negative means

The public synthetic fixture uses the explicit test profile: LUKS1, AES-256-CBC-ESSIV:SHA-256, PBKDF2-SHA1, and 105,474 keyslot iterations. The new CPU checker verifies the LUKS master-key digest.

The selected stock Hashcat v7.1.2 mode 29511 uses a different acceptance test. Its reused kernel decrypts the first payload sector and reports sufficiently low entropy as a potential hit. A correct password can therefore be missed when that plaintext sector has high entropy. Confirming reported hits does not repair such missed hits. See Hashcat module 29511 [3] and the entropy test [4].

Formatting alone is not a sufficient end-to-end fixture: luksFormat writes the header and keyslots, not an encrypted data payload. An unwritten region of zero ciphertext is different from encrypted zero plaintext. See the cryptsetup luksFormat manual [5]. The synthetic fixture generator^{E11} provides encrypted payload and known-password controls.

The campaign consequently labels its Hashcat negatives as payload-entropy results, explicitly not digest-equivalent results. Imported historical coverage also retains its legacy evidence policy. Exact subtraction preserves what that policy accepts; it does not strengthen the underlying evidence. The fact that the volume once held files is insufficient to establish the entropy of the specific sector tested.

7 What the public release demonstrates

The public core tests^{E12} compare output with small exhaustive exact oracles, randomized revisions, tie handling, overlapping explanations, partial history, malformed plans, and deep rank access. The worked example is separately reproduced by the public verifier^{E1}. Counts above JavaScript’s safe-integer range are carried as exact integers or decimal strings, not rounded doubles.

The initial public release passed the following local suites on an Apple M1:

Table 5: 139 local tests in the initial public release.

Suite	Tests	Scope
Ordinary generated tests	104	Ranking, plans, revisions, receipts, CLI and local editor API
Independent large-recipe oracles	4	Bounded synthetic campaign and recipe checks
LUKS reference controls	16	Generated cryptographic fixtures and independent confirmation
GPU orchestration tests	15	Mocked Hashcat behavior; no GPU qualification

Clean-checkout Linux and macOS CI also passed the ordinary and large-recipe suites, 108 tests per platform. The initial source commit is `a8e5934`. The public CI record and the shipped tests make this software validation inspectable. Optional cryptographic suites have additional dependencies and are not part of that CI run.

These results establish the tested behavior, not universal proof of correctness. Symbolic fixtures do not constitute physical password checks. Mocked GPU tests do not qualify actual hardware, establish throughput, or eliminate the entropy checker’s false-negative limitation. The separate synthetic hardware and campaign qualification commands are documented in the adapter guide^{E13}.

Project-specific personal models, historical completion snapshots, and private recovery reports are not supplied as public evidence. No performance or compression claim in this edition depends on those unpublished artifacts.

8 Prior work and the contribution to evaluate

Finite-language compression and automata operations are established techniques. Daciuk, Mihov, Watson, and Watson describe incremental construction of minimal acyclic finite-state automata. Their work is relevant background for shared finite-language representations; this implementation is not claimed to be their algorithm. See Daciuk et al. 2000 [6].

OpenFst provides weighted finite-state machinery including determinization, minimization, and language difference. This project does not claim to invent those operations. See the OpenFst quick tour [7].

Likewise, the PCFG and OMEN work cited earlier establishes prior probability-guided password generation. This initial literature review does not establish that existing tools lack every form of revision handling or duplicate suppression.

The contribution to evaluate is the integration of exact summed string ranking, immutable ordinal plans, symbolic exclusion across revisions, and durable coverage tied to explicit checking policies. Demonstrating that integration is useful does not by itself establish research novelty or superiority.

Further evaluation needs additional reproducible public model fixtures, comparisons against alternative representations and enumerators, resource curves under increasing overlap and history fragmentation, and independent review of the correctness arguments. A successful authorized recovery would be an additional case result, not a prerequisite for those evaluations.

9 Remaining limits and future work

The most consequential present limitation is the gap between an exact candidate set and a checker that can miss a valid password. A digest-verifying GPU backend would address the documented entropy acceptance mechanism. It would still need independent correctness and lifecycle qualification.

A stronger backend would strengthen new checks. Retaining prior entropy-based exclusions would remain a separate acceptance decision: a password already excluded would not automatically be revisited. If that policy is withdrawn, eligibility must be rebuilt from the full model using only evidence accepted under the revised policy. No automatic target-identity or checker-policy migration is established here.

Other work should follow measured needs:

- **Preparation growth.** Measure weighted-state, score-band, and intermediate node growth on difficult models and fragmented histories. Disk-backed storage may change the memory tradeoff but cannot promise infinite histories or remove state explosion.
- **Distributed execution.** A future coordinator would need durable assignment ownership, retry and duplicate semantics, target and plan identity, and result validation. Neither SQLite being a demonstrated bottleneck nor signed leases alone solving trust-worthy execution has been established.
- **GPU candidate generation.** Moving graph traversal onto the GPU could reduce transfer and CPU costs for suitable workloads. Its value depends on actual bottlenecks, graph structure, and target checking cost. No improvement has been measured here.
- **Weight sensitivity.** Compare plausible profiles and identify which early choices depend heavily on unsupported priors. Numerical exactness cannot resolve uncertainty in the original recollection.

No maximum LUKS guesses-per-second figure is asserted. Hardware, runtime, target parameters, batch size, spooling, startup, confirmation, and durable accounting all affect useful throughput.

10 Questions for the design review

Before adopting a new model, the reader should be able to answer:

1. Which admitted constructions follow from observations, and which are proposed alternatives? What possibilities remain outside support?
2. Do branch weights express intended beliefs after accounting for branch size, overlap, and correlated choices?
3. Do the first affordable candidate batches make sense under several plausible allocations, and is the prepared plan complete or bounded?
4. Can every excluded string be traced to accepted evidence for the correct target and checker policy?
5. Is the residual risk of entropy-based negative results understood separately from confidence in the graph subtraction?
6. What measured throughput and failure behavior will justify the first campaign on its actual execution host?

These are review decisions, not invitations to add every proposed feature. The draft leaves the subjective weights unresolved while making the engine's behavior, evidence boundaries, and known execution limitation explicit.

Evidence and reproduction notes

This public edition retains the mathematics, architecture, and limitations of the private design review. It removes personal recollections, private model and history counts, private measurements, and local evidence paths. References below point to shipped source and synthetic checks; they are not personal evidence.

From a fresh checkout:

```
make build
make test test-large
python3 tools/verify_whitepaper.py . /tmp/cems-paper-example
```

The example destination must not already exist. The verifier generates tiny model and plan files, records their exact rows and scores, and exercises full compilation, simulated subtraction, and bounded preparation. It invokes no checker and creates no accepted recovery coverage. Use Python 3.11 or later; set Make's PYTHON explicitly if the system default is older.

With the optional crypto dependencies installed, run:

```
make test-luks1 test-gpu-handoff
```

Build this document using `make whitepaper`. Typst and the shipped vector pipeline figure are sufficient; Graphviz is needed only to edit/regenerate that figure. The PDF is committed alongside its source so readers need no typesetter.

References

- [1] M. Weir, S. Aggarwal, B. de Medeiros, and B. Glodek, “Password Cracking Using Probabilistic Context-Free Grammars,” in *2009 IEEE Symposium on Security and Privacy*, 2009, pp. 391–405. doi: 10.1109/SP.2009.8.
- [2] M. Dürmuth, F. Angelstorff, C. Castelluccia, D. Perito, and A. Chaabane, “OMEN: Faster Password Guessing Using an Ordered Markov Enumerator,” in *Engineering Secure Software and Systems*, in Lecture Notes in Computer Science, vol. 8978. Springer, 2015, pp. 119–132. doi: 10.1007/978-3-319-15618-7_10.
- [3] Hashcat contributors, “Hashcat v7.1.2 Module 29511.” Accessed: Oct. 02, 2026. [Online]. Available: https://github.com/hashcat/hashcat/blob/v7.1.2/src/modules/module_29511.c
- [4] Hashcat contributors, “Hashcat v7.1.2 LUKS Verification Kernel.” Accessed: Oct. 02, 2026. [Online]. Available: <https://github.com/hashcat/hashcat/blob/v7.1.2/OpenCL/m14611-pure.cl#L335-L350>
- [5] cryptsetup contributors, “cryptsetup-luksFormat(8).” Accessed: Oct. 02, 2026. [Online]. Available: <https://man7.org/linux/man-pages/man8/cryptsetup-luksFormat.8.html>
- [6] J. Daciuk, S. Mihov, B. W. Watson, and R. E. Watson, “Incremental Construction of Minimal Acyclic Finite State Automata,” *Computational Linguistics*, vol. 26, no. 1, pp. 3–16, 2000, [Online]. Available: <https://aclanthology.org/J00-1002/>
- [7] OpenFst contributors, “OpenFst Quick Tour.” Accessed: Oct. 02, 2026. [Online]. Available: https://github.com/google-research/openfst/blob/main/docs/quick_tour.md

Appendix A. Public source and verification index

Paths are relative to this public repository. The entries are implementation and reproducibility references, not additional published literature.

E1. public example verifier

tools/verify_whitepaper.py

E2. model semantics

src/emitter_v1/model.py

E3. rational graph export

src/swg.py

E4. native compilation and plan validation

src/native/core_checked.cpp

E5. workspace preparation

src/native/prepare_workspace.cpp

E6. GPU backend

src/gpu_backend.py

E7. spool adapter

src/native/hex_spool.cpp

E8. acknowledgment validation

src/runner.py

E9. GPU worker

src/gpu_worker.py

E10. checkpoint controller

src/checkpoint_runner.py

E11. synthetic fixture generator

src/luks1_fixture.py

E12. public core tests and exact symbolic views

tests/test_core.py; tests/test_recipe_views.py; tests/test_history_campaign.py

E13. adapter guide

docs/ADAPTERS.md

E14. model-bundle workflow

src/model_workflow.py